<Configuration>
<MappedFolders>
<MappedFolder>
<HostFolder>D:\FastApiFoundry-Docker-Sandbox</HostFolder>

<SandboxFolder>C:\Users\WDAGUtilityAccount\Desktop\FastApiFoundry</SandboxFolder>
<ReadOnly>true</ReadOnly>
</MappedFolder>
</MappedFolders>
<Networking>Enable</Networking>
</Configuration>

В таком сценарии запуска как повлияют взаимные изменения в коде. Если я меняю в песочнице - это влияет на локальный код? А нАоборот?

В этой конфигурации поведение строго определяется направлением маппинга:

```xml
<MappedFolder>
  <HostFolder>D:\FastApiFoundry-Docker-Sandbox</HostFolder>
  <SandboxFolder>C:\Users\WDAGUtilityAccount\Desktop\FastApiFoundry</SandboxFolde
```

XML

1. Влияние изменений из песочницы на локальную систему

Не влияют вообще.

Причина:

- `ReadOnly=true` означает, что папка в песочнице является **только для чтения**
- Windows Sandbox монтирует host → sandbox как read-only bind

Следствие:

- Любые изменения в `C:\Users\WDAGUtilityAccount\Desktop\FastApiFoundry`
  - не могут быть записаны обратно в `D:\FastApiFoundry-Docker-Sandbox`
  - либо запрещаются системой, либо откатываются на уровне файловой виртуализации

2. Влияние изменений на локальный код (host → sandbox)

Есть, но только в одну сторону и только до перезапуска песочницы.

- Если ты меняешь файлы на хосте (`D:\FastApiFoundry-Docker-Sandbox`)
- Эти изменения:
  - становятся видны в песочнице
  - при следующем доступе к файлу в sandbox могут подтянуться (обычно почти сразу, так как это live mount)

3. Влияние изменений внутри песочницы (sandbox → host)

Нет. Полностью изолировано.

Даже если бы `ReadOnly=false` , изменения всё равно:

- не переносятся автоматически назад
- исчезают после закрытия песочницы

Песочница всегда работает как disposable environment.

4. Итоговая модель данных

Можно представить так:

```
HOST (D:\FastApiFoundry-Docker-Sandbox)
        ↓  (read-only mount)
SANDBOX (C:\Users\WDAGUtilityAccount\Desktop\FastApiFoundry)
```

- Host → Sandbox: ✅ видно (read-only копия/маппинг)
- Sandbox → Host: ❌ невозможно
- Sandbox state → persistent: ❌ отсутствует

5. Практический вывод

- Песочница = безопасная среда для запуска/экспериментов
- Локальный код = источник истины
- Любые изменения нужно делать на host, если ты хочешь их сохранить

Если хочешь, могу ↪ разобрать вариант с `ReadOnly=false` и объяснить, почему даже там обратная синхронизация работает не так, как ожидается.

Мне плохо readnoly v pesohnice - я буду проверять развертывание

Если ты планируешь проверять развёртывание, `ReadOnly=true` действительно становится ограничением.

Что именно ломается при `ReadOnly=true`

Ты фактически получаешь:

- код можно **читать**
- но нельзя:
  - менять конфиги
  - писать артефакты сборки
  - генерировать `.env`, `.cache`, `node_modules`, `__pycache__`
  - выполнять миграции БД (если они пишут файлы)
  - логировать в файловую систему проекта

Для деплоя это критично, потому что почти любой pipeline в итоге что-то пишет на диск.

Что будет лучше для теста деплоя

Вариант 1 — `ReadOnly=false` (самый прямой)

```XML
```

Что получаешь:

- полноценная рабочая среда

- можно собирать, запускать, менять конфиги
- можно симулировать реальный deploy

Минус:

- изменения в песочнице НЕ попадут обратно в host автоматически (это нормально)

Вариант 2 — гибрид (часто лучший)

Если хочешь контроль:

- host → sandbox: код
- sandbox → host: ничего

И ты:

- работаешь в sandbox как в staging
- но фиксишь код только на host

Вариант 3 — отдельная writable рабочая директория в sandbox

Иногда делают так:

```xml
<MappedFolder>
  <HostFolder>D:\FastApiFoundry-Docker-Sandbox</HostFolder>
  <SandboxFolder>C:\Users\WDAGUtilityAccount\Desktop\repo</SandboxFolder>
```

XML

И внутри sandbox:

- код читается и правится
- но ты осознанно понимаешь, что это “throwaway workspace”

Рекомендация под твой кейс (deploy testing)

Тебе нужен:

- сборка
- запуск контейнеров/серверов

- генерация файлов
- возможно Docker build / compose

👉 значит:

`ReadOnly=false` — практически обязательный вариант

Важный нюанс

Даже с `ReadOnly=false`:

- песочница остаётся **одноразовой**
- любые изменения исчезают при закрытии
- это не dev sync среда, а ephemeral staging

Если хочешь более “правильную” схему

Могу предложить архитектуру:

- sandbox = CI-like runner
- host = source of truth
- автоматический sync через PowerShell (robocopy / git hooks)
- запуск деплоя как reproducible pipeline

Скажи, если хочешь — соберу тебе такую схему под FastAPI + Docker.

А сеть? Мне нужна сеть

В Windows Sandbox сеть управляется отдельно и по умолчанию включена:

XML

Что даёт `Networking=Enable`

В песочнице появляется полноценный сетевой стек:

- доступ в интернет (HTTP/HTTPS, DNS)
- доступ к внешним API
- `git clone`, `pip install`, `npm install`
- доступ к Docker registry (если Docker внутри sandbox)
- обращение к локальной сети (частично зависит от политики хоста)

Важные ограничения

1. Изоляция от хоста

Sandbox **не является частью твоей локальной сети как обычная VM**:

- ты не можешь обращаться к хосту через `localhost`
- `127.0.0.1` внутри sandbox = сам sandbox
- прямой доступ к `D:\` есть только через mapped folder

---

2. Доступ к локальным сервисам хоста

Иногда возможно:

- через IP хоста (например, `192.168.x.x`)
- но:
  - firewall Windows может блокировать
  - sandbox NAT-слой может ограничивать входящие соединения

---

3. Входящие соединения в sandbox

По умолчанию:

- ❌ нельзя подключиться *внутрь* sandbox извне
- ❌ sandbox не является сервером для хоста/локальной сети

Практическое поведение для dev / deploy testing

Работает нормально:

- обращение к внешним API

- скачивание зависимостей

ChatGPT Free offer ×

- Docker pulls / builds (если Docker установлен)

Часто ломается:

- webhooks, которые должны стучаться в sandbox
- локальные callback URL (OAuth redirect на `localhost`)
- интеграция с сервисами на host

Типичная ошибка в твоём сценарии

Если ты тестируешь FastAPI:

- ты запустишь `uvicorn` в sandbox
- он будет доступен **только внутри sandbox**

👉 с хоста ты его не откроешь через `localhost:8000`

Если тебе нужен реальный dev-server сценарий

Есть 3 варианта:

1. Port-forward через host IP (нестабильно)

Используешь IP хоста и открытый порт

2. Туннель (лучший вариант)

- `ngrok`
- `cloudflared tunnel`

3. Docker внутри sandbox

Полная изоляция + сеть внутри контейнеров

Итог

- `Networking=Enable` — тебе нужно, и он включён правильно
- sandbox = NAT-клиент, не полноценный узел сети

- входящие соединения — проблема архитектурно

Если скажешь, что именно ты хочешь проверять в деплое (FastAPI + Docker + DB + frontend), я могу разложить тебе точную сетевую схему: что куда должно ходить и где sandbox реально подходит, а где начнёт ломать сценарий.